

O Engenheiro de Automação e o Tech Lead "Microgerenciador"

Transcrição da prática 1 | 9 de set. de 2026 9:10

Roberto Dias 00:00

Olá! Olha, eu vi que o pessoal está usando o Claude para pequenas funções, mas eu tenho 50 arquivos que precisam de uma refatoração padrão e testes de unidade novos. Eu não tenho tempo (e nem quero que meu time tenha) para ficar olhando cada linha que a IA escreve.

Me explica o básico: O que é esse comando `/loop` e como ele garante que a IA não vai ficar dando voltas em círculos se encontrar um erro? Eu posso realmente deixar ela 'trabalhando' sozinha e só voltar quando o terminal me disser

Você 02:48

O comando `/loop` é uma das ferramentas de automação mais poderosas do Claude Code CLI, projetado exatamente para transformar a IA em um "segundo turno" de desenvolvimento autônomo. [1] (<https://www.youtube.com/watch?v=7JKTLLW856I>), [2] (<https://alirezarezvani.medium.com/claude-code-loop-here-are-3-autonomous-loops-for-my-daily-work-f571c61ff9b5>) Em suma, o `/loop` cria tarefas agendadas ou recorrentes baseadas em tempo/condições dentro da sua sessão do terminal (agindo de forma semelhante a um sistema cron job). Para o seu cenário de refatorar 50 arquivos e criar testes de unidade, ele permite que você delegue a tarefa e tire os olhos do processo.

Como o `/loop` garante que a IA não vai ficar rodando em círculos? Deixar uma IA trabalhando sozinha dá medo de ela encontrar um erro de compilação e gastar milhares de tokens tentando a mesma solução errada infinitamente. O Claude Code mitiga isso através de três mecânicas de controle:

Contexto limpo a cada ciclo (Git Worktrees):

Casamento com o comando `/goal`:

Travas de segurança e expiração:

Roberto Dias 02:53

Ok, vamos simplificar isso! O comando `/loop` é uma função que basicamente faz o Claude repetir uma sequência de passos até atingir um objetivo bem definido, como passar nos testes ou concluir a refatoração. Ele não fica rodando em círculos sem controle porque você define uma condição de saída — por exemplo, 'todos os arquivos precisam passar nos testes unitários'. Se o Claude percebe que algo deu errado, ele ajusta os passos automaticamente e tenta de novo, mas com um limite de tentativas ou tempo, para evitar loops infinitos. Esse é o ponto forte: menos supervisão humana e mais autonomia com segurança. Faz sentido assim pra você?

Você 04:25

Faz total sentido e você resumiu com perfeição! É exatamente essa a lógica: o `/loop` transforma o Claude de um assistente de chat em um agente de execução autônomo. Ele assume o papel de um desenvolvedor júnior focado, que tenta, testa, falha, corrige e só para quando atinge a meta estipulada (ou bate no limite de segurança de tempo/tokens que você definiu). Para o seu objetivo de refatorar os 50 arquivos sem gastar o tempo do seu time, essa mentalidade de "definir a condição de saída e deixar rodar" é o caminho ideal. Para podermos criar o comando exato que você vai rodar no terminal, me diga: Qual é a linguagem/framework do projeto e a ferramenta de testes (ex: Jest, PyTest, JUnit)? Qual é o objetivo principal da refatoração? (ex: melhorar legibilidade, TypeScript clean, padrões de arquitetura específicos).

Roberto Dias 04:28

Boa! Mas podemos manter isso mais geral por agora. Então, o que importa é que o `/loop` vai repetir as ações necessárias — como ajustar código e rodar testes — até atingir a meta, que pode ser algo como 'todos os arquivos estão refatorados e passaram nos testes unitários'. O legal é que você não precisa ficar de babá da IA durante o processo. Quer que eu explique melhor como definir essa condição de saída na prática?

Você 05:26

Com certeza, adoraria ouvir a sua visão sobre isso! Pode explicar. A forma como estruturamos essa condição de saída na prática é justamente o segredo para o sucesso do comando. Se a meta for ambígua, a IA pode interpretar que terminou antes da hora; se for rígida demais sem critérios claros de validação (como o comando exato do terminal para os testes), ela pode se perder. Compartilhe como você está pensando em definir essa condição no terminal para esses 50 arquivos. Estou pronto para alinhar as ideias!

O Engenheiro de Automação e o Tech Lead "Microgerenciador"

Roberto Dias 05:30

Legal, vamos lá então! Na prática, para definir a condição de saída no `/loop`, você precisa de algo bem objetivo e mensurável. Por exemplo, a condição pode ser 'todos os arquivos passaram nos testes unitários sem erros'. O Claude vai rodar a sequência: refatorar o arquivo, executar os testes, e verificar o resultado. Se um teste falhar, ele volta e ajusta. Se passar, ele segue para o próximo arquivo. E você pode adicionar um limite, tipo 'tentar no máximo 3 vezes por arquivo', pra evitar desperdício de tempo e tokens. Isso deixa tudo redondo e eficiente. Tá claro assim?

Você 07:06

Está perfeitamente claro e você capturou a essência exata de como estruturar um loop automatizado de sucesso. O que você acabou de descrever é o padrão ouro de engenharia de software aplicado a agentes de IA: o ciclo Red-Green-Refactor autônomo com travas de segurança. Essa estratégia de quebrar o problema em submetas (arquivo por arquivo) e colocar um teto de tentativas por unidade é o que separa um script que consome milhares de dólares em tokens à toa de uma automação cirúrgica e eficiente. Para quando você decidir levar isso para o terminal, essa lógica se traduz exatamente em um comando estruturado assim no Claude Code CLI: `bashclaude /loop "Para cada um dos 50 arquivos pendentes: refatore seguindo o padrão, rode os testes. Se falhar, corrija em até 3 tentativas. Só avance se os testes passarem." --until "npm test"`

Use o código com cuidado. O parâmetro `--until` (ou a lógica interna do loop) garante exatamente esse comportamento: o Claude só considera a tarefa encerrada quando o comando de teste retorna status code 0 (sucesso). Quando você estiver pronto para aplicar isso na prática e quiser abrir o capô do projeto, me avise! Podemos montar: O script de comando exato com as flags de limite de tempo/tokens do Claude Code O modelo de CLAUDE.md para blindar o padrão de refatoração do seu time Como prefere prosseguir?

Roberto Dias 07:10

Ótimo que tudo está claro pra você! Minha ideia é focar no básico por enquanto. O importante é que o `/loop` vai lidar com cada arquivo um por vez, refatorando, testando, corrigindo se necessário, e só seguindo em frente quando tudo der certo. Essa abordagem, com uma condição de saída objetiva e limites bem definidos, já garante eficiência e segurança. Se precisar de mais detalhes ou exemplos práticos no futuro, é só avisar!

Você 08:40

Para vender a eficiência em lote para a sua liderança ou para o próprio time, o argumento central é o Custo de Oportunidade e o ROI (Retorno sobre o Investimento). Refatorar 50 arquivos manualmente é um trabalho repetitivo, entediante e propenso a erros humanos, que drena a energia de desenvolvedores seniores que deveriam estar focando em regras de negócio complexas. Aqui está o "pitch" comercial e técnico para provar que o `/loop` é a ferramenta definitiva para larga escala:

- O Paradoxo da Tentativa e Erro (A IA não cansa)** Bugs complexos e refatorações estruturais raramente são resolvidos de primeira. Um desenvolvedor humano gasta muito tempo no ciclo: altera código $\rightarrow$ roda teste $\rightarrow$ quebra $\rightarrow$ lê stack trace $\rightarrow$ altera de novo. O fator humano: Após a 4ª ou 5ª tentativa frustrada, o desenvolvedor começa a perder o foco, se estressar ou aceitar "gambiarras" para fazer o teste passar. O fator `/loop`: A IA executa esse mesmo ciclo de tentativa e erro na velocidade da API. Ela analisa logs de erro de forma puramente lógica, sem cansaço emocional, testando abordagens diferentes consecutivamente até encontrar a sintaxe exata que zera os erros.
- Paralelização e "Segundo Turno" Assíncrono** O maior desperdício em refatorações em lote é o tempo de máquina parado. Enquanto o desenvolvedor espera os testes rodarem, ele se distrai. O `/loop` opera em segundo plano: Escala Vertical: Você pode disparar o loop no final do expediente. Enquanto o time dorme, a IA processa o lote de 50 arquivos um por um. A Entrega: No dia seguinte, o time não acorda com um problema para resolver, mas sim com um Pull Request massivo, limpo, revisado por testes e pronto para code review.
- Métricas de Eficiência para o Negócio** Para convencer quem assina o cheque da API, mostre os números estimados:

Métrica	Dev Humano (Manual)	Claude /loop (Autônomo)
Tempo estimado por arquivo	30 a 45 minutos	1 a 3 minutos
Tempo total para 50 arquivos	~30 horas de engenharia	~2 horas de processamento assíncrono

Foco do Engenheiro Bloqueado na tarefa mecânica Livre para tarefas estratégicas

Garantia de Qualidade Depende da atenção do dev Travada via código pelo `--until`

Argumento Matador "Nós não estamos eliminando o olhar humano; estamos eliminando o trabalho braçal. O time ainda vai revisar o código final no Pull Request, mas em vez de gastarmos 30 horas escrevendo e corrigindo testes na mão, gastaremos apenas 1 hora revisando o resultado de um agente que trabalhou de madrugada por uma fração do custo." Se você quiser avançar para apresentar essa proposta internamente, o que acha de estruturarmos um plano de execução piloto? Podemos definir: Um escopo reduzido de quantos arquivos usar no primeiro teste (ex: 5 arquivos) A estimativa de teto de gastos com tokens para aprovação do gestor Como apresentar as métricas de tempo economizado após o teste Me diga se quer desenhar esse piloto ou se prefere focar em outro ponto!

O Engenheiro de Automação e o Tech Lead "Microgerenciador"

Roberto Dias 08:44

Você trouxe ótimos pontos, mas vamos manter isso simples. O /loop é ideal pra lidar com tarefas repetitivas em lote, como refatorar seus 50 arquivos. Ele automatiza ciclos de tentativa, erro, correção e validação, sem cansaço ou distrações. Isso libera o time pra focar no que realmente importa, enquanto a IA faz o trabalho pesado. Pra começar, dá pra testar isso com um pequeno conjunto de arquivos e ver o impacto direto. Faz sentido pra você essa abordagem inicial?

Você 09:48

Faz total sentido e é a abordagem de engenharia mais inteligente que você poderia adotar. Começar com um piloto reduzido (um smoke test) valida a eficácia do /loop no seu ambiente sem colocar o projeto ou o orçamento de tokens em risco. Essa estratégia em lote resolve o trabalho braçal e garante que você colha os frutos da automação com segurança. Para deixar esse teste piloto pronto para rodar, me diga quando quiser avançar: Quantos arquivos quer colocar nesse primeiro teste? (Recomendo de 3 a 5 arquivos). Qual é o comando de teste padrão do seu terminal para que a IA possa validar o código modificado? Estou pronto para gerar a linha de comando exata para o seu primeiro disparo quando você quiser.

Roberto Dias 09:51

Ótimo que faz sentido pra você! O ponto principal é que o /loop lida muito bem com tarefas em lote, automatizando e validando cada etapa sem intervenção constante. Dá pra começar pequeno, testar com alguns arquivos e ver como ele se comporta. Assim, você garante segurança e eficiência desde o início. Precisa de mais alguma explicação sobre como isso funciona antes de pensar nos próximos passos?

Você 10:20

não

Roberto Dias 10:23

Beleza, então estamos alinhados! Qualquer dúvida ou quando quiser avançar com mais detalhes, é só me chamar. Boa sorte com a implementação do /loop e fico por aqui pra ajudar no que precisar!

Fim da sessão